

Operating Systems

Lab experience - pipes and named pipes

Mirco Marchetti, Riccardo Lancellotti

SECloud research group
University of Modena and Reggio Emilia

AY 2025/26

Goals of this lab experience:

- Getting used to pipes
- Getting used to named pipes (FIFOs)

Rock, paper, scissors!

- Review the solution to the exercise 4 of the previous lab experience
Moodle → Esercitazioni e dimostrazioni → Laboratorio shared
memory e pipe → solution-4.c
- Read it, understand how it works, compile and run it, check that it works as expected

Exercise 1: rock, paper, scissors (1 of 2)

- Modify solution-4.c to create a new program that lets two children play the rock-paper-scissors game against each other, with the parent acting as a referee.
- Each child process must install a signal handler for SIGUSR1 and then suspend its execution using `pause()`, waiting for the parent to start a new round (as in solution-4.c).
- The parent process implements a game that proceeds in multiple rounds. At the beginning of each round, the parent waits for the user to press ENTER, then sends SIGUSR1 to both children to signal the start of the round (as in solution-4.c).
- Upon receiving SIGUSR1, both children randomly select one of the three possible moves (rock, paper, or scissors) and write a message to the brother including their move and their process identifier.

Exercise 1: rock, paper, scissors (2 of 2)

- Each children reads the message from the other child, determines the winner of the round based on the rules of the game, and sends a message to the parent through its pipe indicating whether it won, lost, or tied.
- The parent reads the messages from both children, determines the outcome of the round, updates the score for each child, and displays the current scores. The game continues until one of the two children reaches a predefined score (e.g., 5 points). At that point, the parent declares the winner.

Exercise 1: hints

- Start from solution-4,c of the previous lab experience, and modify it to implement the new game logic. You can reuse the existing code for process creation, signal handling, and children-to-parent inter-process communication through pipes.
- You will need **two additional pipes** for the children to communicate with each other. Make sure to close the appropriate pipe descriptors in each process to avoid deadlocks and ensure proper communication.
- To improve readability, you can define constants for the moves (e.g., ROCK=0, PAPER=1, SCISSORS=2) and for the outcomes (e.g., LOST=0, WON=1, TIE=2).
- Use two structs to define the messages. One struct for communication among children and another struct for communication between children and parent.

Rock, paper, scissors! But with named pipes!

- Within Moodle → Esercitazioni e dimostrazioni → Laboratorio pipe e named pipe you will find the following programs:
 - mkfifo-server-example.c
 - mkfifo-client-example.c
- They implement a simple "server" and a "client" that communicate through a named pipes (FIFO).
- The server creates all the FIFOs, two client instances use these FIFOs to communicate.
 - First clients exchange a message among themselves
 - Then each client sends a message to the server
- Read them, understand how it works, compile and run them, check that they work as expected
 - Use three terminals, first start the server, then start the two clients using parameters 1 and 2.

- You might have noticed that the server and clients follow the same communication scheme as the solution to exercise 1 of the previous part, but they use named pipes instead of anonymous pipes.
- Your task is to modify the solution to exercise 1 to use named pipes instead of anonymous pipes for all communication between the parent and the children, and between the children themselves.

Exercise 2: hints

- Reuse the code of `mkfifo-server-example.c` and `mkfifo-client-example.c` to implement the new communication logic using named pipes. You can reuse the existing code for process creation and signal handling from your solution to exercise 1.
- **Important:** opening a named pipe for reading will block until another process opens the same pipe for writing, and vice versa. To avoid permanent blocking (deadlock) let the server open all the FIFOs for reading and writing before starting the game loop. This way, when a client opens a FIFO for reading or writing, it will not block because the server has already opened it in the opposite mode.
- **Very important:** named pipes are still **unidirectional**, opening them in read-write mode is only a trick to avoid blocking, but you still need to use separate FIFOs for each direction of communication.

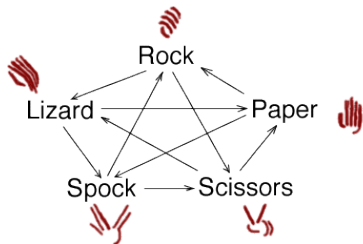
- The game server and the client processes are no longer related! They are completely independent processes that communicate through the FIFOs. How can the server know the PID of the two clients to send signals to them?
- Before starting the game loop, each client can write a `registration` message with its PID to the server.
- The server read the registration message from each client, extracts the PID, and stores it in a variable. Then, when the server needs to signal the clients to start a new round, it can use the stored PIDs to send `SIGUSR1` to both clients.

Rock, paper, scissors, lizard, Spock!

Exercise 3: rock, paper, scissors, lizard, Spock

- Extend the solutions of exercise 2 to implement the rock-paper-scissors-lizard-Spock game, which is an expansion of the traditional rock-paper-scissors game with two additional moves: lizard and Spock. The rules of the game are as follows:
 - Scissors cuts paper
 - Paper covers rock
 - Rock crushes lizard
 - Lizard poisons Spock
 - Spock smashes scissors
 - Scissors decapitates lizard
 - Lizard eats paper
 - Paper disproves Spock
 - Spock vaporizes rock
 - Rock crushes scissors

Rock, paper, scissors, lizard, Spock



Scissors cuts Paper covers Rock crushes
Lizard poisons Spock smashes Scissors
decapitates Lizard eats Paper disproves
Spock vaporizes Rock crushes Scissors.

<https://www.samkass.com/theories/RPSSL.html>
<https://www.youtube.com/watch?v=pIpmITBocfM>